

Data Structures and Algorithms
23CSE203
Lab Manual

SUBMITTED BY:

NAME:	M. KARTHIK REDDY
ROLL NO:	CSE25037
YEAR / SEM / SECTION	2 / 3 / CSE-A
CREDITS	5

SUBMITTED TO:

NAME:	Dr. RAJKUMAR BATCU
DEPARTMENT	SCHOOL OF COMPUTING
DESIGNATION	ASST. PROFFESOR (SI grade)

S NO:	TITLE	DATE	REMARKS
1	WEEK 1: 1a: Recursive countdown from n 1b: Recursive calculation of compound growth factor 1c: Recursive search for emo id in list 1d: Recursive factorial calculation 1e: Recursive fibonacci series	23-07-26	
2	WEEK 2: 2a: Linear Search Program 2b: Binary Search Program	30-07-27	
3	WEEK 3: 3a: Bubble Sort 3b: Insertion Sort 3c: Selection Sort 3d: Merge Sort 3e: Quick Sort	06-08-26	
4	WEEK 4: 4a: Singly Linked List 4b: Doubly Linked List 4c: Circular Linked List	13-08-26	
5	WEEK 5: 5a: Stack Using Array 5b: Stack Using Linked List	27-08-26	
6	WEEK 6: 6a: Queue Using Array 6b: Queue Using Linked List 6c: Circular Queue	03-09-26	
7	WEEK 7: 7a: Binary tree using arrays 7b: Binary tree using linked list	24-09-26	
8	WEEK 8: 8a: Binary search tree using arrays 8b: Binary search tree using linked list	08-10-26	
9	WEEK 9: 9a: AVL tree using arrays 9b: AVL tree using linked list	22-10-26	
10	WEEK 10: 10a: Hashing using arrays 10b: Hashing using linked list	29-10-26	

LAB 1

1. AIM:
A rocket launch system displays a countdown. Use no loops. Write a Python program to display the countdown from n to 1, followed by “Launch”.

PROGRAM:

```
def countdown(n):
    print(n)
    if n==0:
        return 0
    else:
        return countdown(n-1)

n=int(input("enter number"))
print(countdown(n))
```

OUTPUT:

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
======= RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/1.py =======
enter number9
9
8
7
6
5
4
3
2
1
0
0
>>>
```

EXPLANATION:

- Defines a recursive countdown() function that prints numbers in decreasing order.
- Prints the current value of n before making the recursive call.
- When n becomes 0, the recursion stops.
- Otherwise, the function calls itself with n-1.
- Takes the starting number as input and displays the countdown.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Countdown does not stop	Check the condition when n reaches 0
2	Program stops for a very large number	Use a loop instead of recursion
3	Negative numbers are printed unexpectedly	Allow only non negative starting values

2. AIM:

Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power), where p = principal growth factor and n = number of years.

PROGRAM:

```
def power(p,n):
    if n<=0:
        return 1
    else:
        return p*power(p,n-1)

p=int(input("Enter prinipal"))
n=int(input("Enter number"))
Print(power(p,n))
```

OUTPUT:

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/2.py ==========
Enter prinipal10000
Enter number5
100000000000000000000
>>>
```

EXPLANATION:

- Defines a recursive power() function to calculate \(p^n\).
- Returns 1 when n <= 0, which is the base case.
- Otherwise, multiplies p by power(p, n-1) repeatedly.
- Takes the base (p) and exponent (n) as input.
- Calls the function and prints the calculated power.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Incorrect result for negative exponent	Add separate handling for negative exponents
2	Program becomes slow for large exponent	Use an efficient power calculation method
3	Wrong result for exponent zero	Make sure the base case returns 1

3. AIM:

An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(A,k,n=0):
    if n>=len(A):
        return -1
    if A[n]==k:
        return n
    return search(A,k,n+1)

A=[1,2,3,4,5,6,7,8]
print(A)
k=int(input("enter id to search"))

Print(search(A,k))
```

OUTPUT:

```
>>>
================================ RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/3.py ================================
[1, 2, 3, 4, 5, 6, 7, 8]
enter id to search5
4
>>>
```

EXPLANATION:

- Defines a recursive search() function to find the target element in the list.
- Checks each element starting from index 0 using recursion.
- If the target is found, its index is returned.
- If the end of the list is reached, -1 is returned.
- Takes the target value as input and displays the search result.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Target element is not found	Check whether the target exists in the list
2	Recursion stops before checking all elements	Verify the index increment in the recursive call
3	Program stops for a large list	Use an iterative search method

4. AIM:

In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
    if n<=0:
        return 1
    else:
        return n*factorial(n-1)

n=int(input("Enter number"))
print(factorial(n))
```

OUTPUT:

```
>>>
========== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/4.py ==========
Enter number7
5040
>>>
```

EXPLANATION:

- Defines a recursive factorial() function to calculate the factorial of a number.
- Returns 1 when n <= 0, which also handles 0!.
- Otherwise, multiplies n by factorial(n-1) until reaching the base case.
- Takes a number as input from the user.
- Calls the function and prints the factorial result.

ERROR TABLE:

S.No.	Error Obtained	Rectification

1	Incorrect factorial for negative input	Restrict the input to non negative numbers
2	Program stops for a large number	Use an iterative approach
3	Wrong factorial result	Check the recursive multiplication and base condition

5. AIM:
Write a recursive Python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fibonacci(n):
    if n<=0:
        return 0
    elif n==1:
        return 1
    else:
        return fibonacci(n-1)+fibonacci(n-2)


n= int(input("Enter terms"))
for i in range(n):
    print(fibonacci(i))
```

OUTPUT:

```
>>>
    ================================ RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 1/5.py ================================
    Enter terms8
    0
    1
    1
    2
    3
    5
    8
    13
>>>
```

EXPLANATION:

- Defines a recursive fibonacci() function to calculate Fibonacci numbers.
- Returns 0 for n <= 0 and 1 for n == 1.
- For other values, it recursively adds the previous two Fibonacci numbers.
- Takes the number of terms from the user.
- Uses a loop to generate and print each Fibonacci term.
- The sequence starts as 0, 1, 1, 2, 3, 5, ...

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Program runs very slowly	Use an iterative or optimized approach
2	Incorrect sequence is displayed	Check the base conditions for 0 and 1
3	Program stops for large input	Avoid excessive recursive calls

LAB 2

1. AIM:
Write a program to perform linear search on a given input.

PROGRAM:

```
def linearSearch(a,key):
    for i in range(n):
        if a[i]==key:
            return i
    return -1

n=int(input("Enter number of elements"))
a=[]
for i in range(n):
    x=int(input("enter elements"))
    a.append(x)
key=int(input("Enter target element"))
Print(linearSearch(a,key))
```

OUTPUT:

```
>>>
========== RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 2/1.py ==========
Enter number of elements5
enter elements2
enter elements3
enter elements4
enter elements5
enter elements9
Enter target element4
2
>>>
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Takes the target element to be searched.
- linearSearch() checks each element one by one.
- If the target is found, its index is returned.
- If not found, -1 is returned.
- The result is displayed using print().

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Element is not found	Check whether the required element is present in the list
2	Wrong index is displayed	Check the list indexing starting from 0

3	Search stops unexpectedly	Verify the range used in the for loop

2. AIM:

Write a Python program to perform binary search on the given input. Verify by taking sorted array & unsorted array as input.

PROGRAM:

```
def binary_search(a, low, high, key):
    while low <= high:
        mid = (low + high) // 2

        if a[mid] == key:
            return mid
        elif a[mid] > key:
            high = mid - 1
        else:
            low = mid + 1

    return -1

n = int(input("Enter number of elements: "))
if n <= 0:
    print("Number of elements must be positive.")
a = []
for i in range(n):
    x = int(input("Enter element: "))
    a.append(x)
a.sort()
print("Sorted list:",a)
key = int(input("Enter target element: "))
result = binary_search(a, 0, len(a) - 1, key)
if result != -1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
>>>
================================ RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 2/2.py ================================
Enter number of elements: 5
Enter element: 3
Enter element: 6
Enter element: 7
Enter element: 1
Enter element: 9
Sorted list: [1, 3, 6, 7, 9]
Enter target element: 7
Element found at index 3
>>>
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Sorts the list using a.sort() because binary search requires sorted data.
- Calculates the middle element and compares it with the target.
- If the target is smaller or larger, searches only the appropriate half.
- Returns the index if found; otherwise returns -1.
- Displays whether the element is found or not.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Wrong element is returned	Make sure the list is sorted before searching
2	Search gives incorrect index	Check the calculation of the middle index
3	Element is not found	Verify the low and high boundaries

LAB 3

1. AIM:

Write a program to sort the elements using a bubble sort technique. Take an input elements from user

PROGRAM:

```
def bubble_sort(arr):
    n=len(arr)
    for i in range(n-1):
        for j in range(n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1]=arr[j+1],arr[j]
    return arr
num=int(input("Enter number of elements: "))
arr=[]
for i in range(num):
    x=int(input("Enter element: "))
    arr.append(x)
print(arr)
print("Sorted array is: ",bubble_sort(arr))
```

OUTPUT:

```
======= RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/1.py =======
Enter number of elements: 6
Enter element: 6
Enter element: 2
Enter element: 5
Enter element: 9
Enter element: 0
Enter element: 7
[6, 2, 5, 9, 0, 7]
Sorted array is:  [0, 2, 5, 6, 7, 9]
```

EXPLANATION:

- Defines the bubble_sort function and finds the number of elements in the list.
- Compares adjacent elements using two nested loops.
- Swaps the elements if they are in the wrong order.
- Repeats the process until all elements are arranged in ascending order.
- Takes the elements as input from the user and stores them in a list.
- Calls bubble_sort and displays the sorted array.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	IndexError while comparing elements	Check the loop range and array size

2	Elements are not sorted correctly	Check the swapping condition
3	Incorrect number of elements processed	Make sure the entered count matches the elements

2. AIM:
Write a program to sort the elements using a insertion sort technique. Take an input elements from user

PROGRAM:
```
def insertion_sort(arr):
    n=len(arr)
    for i in range(1,n):
        key=arr[i]
        j=i-1
        while j>=0 and arr[j]>key:
            arr[j+1]=arr[j]
            j=j-1
            arr[j+1]=key
    return arr
num=int(input("Enter number of elements: "))
arr=[]
for i in range(num):
    x=int(input("Enter element: "))
    arr.append(x)
print(arr)
print("Sorted array is: ", insertion_sort(arr))
```

OUTPUT:
```
======= RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/2.py =======
Enter number of elements: 5
Enter element: 2
Enter element: 18
Enter element: 7
Enter element: 45
Enter element: 10
[2, 18, 7, 45, 10]
Sorted array is:  [2, 7, 10, 18, 45]
```

EXPLANATION:
- Defines the insertion_sort function and starts from the second element of the list.
- Stores the current element in key and compares it with the previous elements.
- Moves larger elements one position to the right.
- Places key in its correct position in the sorted portion of the list.
- Repeats the process until all elements are sorted.
- Displays the original and sorted arrays.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Elements are not sorted correctly	Check the position where key is inserted
2	IndexError while shifting elements	Check the condition j >= 0
3	Some elements are overwritten	Place key after completing the shifting process

3. AIM:
Write a program to sort the elements using a selection sort technique. Take an input elements from user

PROGRAM:
```
def selection_sort(arr):
    n=len(arr)
    for i in range(n-1):
        min_index=i
        for j in range(i+1,n):
            if arr[j]<arr[min_index]:
                min_index=j
                arr[i] , arr[min_index] = arr[min_index] , arr[i]
    return arr
num=int(input("Enter number of elements: "))
arr=[]
for i in range(num):
    x=int(input("Enter element: "))
    arr.append(x)
print("Sorted array is: ", selection_sort(arr))
```

OUTPUT:

```
======= RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/3.py =======
Enter number of elements: 4
Enter element: 9
Enter element: 4
Enter element: 1
Enter element: 0
Sorted array is:  [0, 1, 4, 9]
```

EXPLANATION:
- Defines the selection_sort function and assumes the first unsorted element as the minimum.
- Compares it with the remaining elements to find the smallest element.
- Stores the position of the smallest element in min_index.
- Swaps the smallest element with the element at the current position.

- Repeats the process until the entire list is sorted.
- Takes input elements and displays the sorted array.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Incorrect sorting of elements	Perform the swap after finding the minimum element
2	Elements are swapped multiple times	Move the swap statement outside the inner loop
3	Minimum element is selected incorrectly	Check the min_index update condition

4. AIM:
Write a program to sort the elements using a merge sort technique. Take an input elements from user

PROGRAM:
```
def merge_sort(a):
    if len(a)>1:
        mid=len(a)//2
        left=a[:mid]
        right=a[mid:]
        merge_sort(left)
        merge_sort(right)
        i=j=k=0
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                a[k]=left[i]
                i+=1
            else:
                a[k]=right[j]
                j+=1
            k+=1
        while i<len(left):
            a[k]=left[i]
            i+=1
            k+=1
        while j<len(right):
            a[k]=right[j]
            j+=1
            k+=1
num=int(input("Enter number of elements: "))
a=[]
for i in range(num):
    x=int(input("Enter element: "))
    a.append(x)
print("Original array:",a)
merge_sort(a)
```

```
print("Sorted array:",a)
```

OUTPUT:

```
======= RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/4.py =======
Enter number of elements: 5
Enter element: 3
Enter element: 5
Enter element: 6
Enter element: 2
Enter element: 0
Original array: [3, 5, 6, 2, 0]
Sorted array: [0, 2, 3, 5, 6]
```

EXPLANATION:

- Defines the merge_sort function and divides the array into two halves.
- Recursively divides each half until single elements remain.
- Compares elements from the left and right halves.
- Merges the elements back into the array in ascending order.
- Continues merging until the complete array is sorted.
- Takes input elements and displays the original and sorted arrays.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Elements are not merged correctly	Check the comparison between left and right arrays
2	Some elements are missing after merging	Check the remaining element loops
3	Incorrect sorted output	Verify the values of i, j and k during merging

5. AIM:

Write a program to sort the elements using a quick sort technique. Take an input elements from user

PROGRAM:

```
def quicksort(a,low,high):
    if low<high:
        i=low
        j=high
        pivot=low
        while i<j:
            while i<=high and a[i]<=a[pivot]:
                i+=1
            while a[j]>a[pivot]:
                j-=1
            if i<j:
                a[i],a[j]=a[j],a[i]
```

```
        a[pivot],a[j]=a[j],a[pivot]
        quicksort(a,low,j-1)
        quicksort(a,j+1,high)

num=int(input("Enter number of elements: "))
a=[]
for i in range(num):
    x=int(input("Enter element: "))
    a.append(x)
print(a)
n=len(a)
quicksort(a,0,n-1)
print("Sorted array is:",a)
```

OUTPUT:

```
======= RESTART: /Users/karthikreddy/Documents/SEM 3/DSA/lab/week 3/5.py =======
Enter number of elements: 5
Enter element: 4
Enter element: 7
Enter element: 2
Enter element: 9
Enter element: 8
[4, 7, 2, 9, 8]
Sorted array is: [2, 4, 7, 8, 9]
```

EXPLANATION:

- Defines a recursive Quick Sort function and selects the first element as the pivot.
- Uses two pointers, i and j, to compare and arrange elements around the pivot.
- Swaps elements when they are on the wrong side of the pivot.
- Places the pivot in its correct position after partitioning.
- Recursively sorts the elements before and after the pivot.
- Takes the elements as input and performs the sorting operation.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	UnboundLocalError for i or j	Keep the partitioning code inside the if block
2	RecursionError for large input	Check the recursive calls and pivot position
3	Sorted array is displayed as None	Print the list after calling quicksort

LAB 4

1. AIM:

Write a python programme to implement a single linked list which could perform the below operations ,by taking an input from the user.

1)create a linked list
2)Insert at beginning
3)Insert at end
4)insert at index
5)Delete by value
6)Delete First Node
7)Delete Last Node
8)Count number of nodes
9)Display/Traverse
10)Exit

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SLL:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        self.head = None
        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)
            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next is not None:
                    temp = temp.next
                temp.next = new_node
        print("Linked List created successfully.")

    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)
```

```
        new_node.next = self.head
        self.head = new_node
        print(f"Inserted {data} at the beginning.")

    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            print(f"Inserted {data} at the end.")
            return
        temp = self.head
        while temp.next is not None:
            temp = temp.next
        temp.next = new_node
        print(f"Inserted {data} at the end.")

    def insert_at_index(self):
        data = int(input("Enter data: "))
        index = int(input("Enter index: "))
        if index < 0:
            print("Invalid index.")
            return
        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            print(f"Inserted {data} at index {index}.")
            return
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Index out of range.")
                return
            temp = temp.next
        if temp is None:
            print("Index out of range.")
            return
        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node
        print(f"Inserted {data} at index {index}.")

    def delete_by_value(self):
        value = int(input("Enter value to delete: "))
        if self.head is None:
            print("List is empty.")
            return
        if self.head.data == value:
            self.head = self.head.next
            print(f"Deleted {value}.")
            return
```

```
        temp = self.head
        while temp.next is not None:
            if temp.next.data == value:
                temp.next = temp.next.next
                print(f"Deleted {value}.")
                return
            temp = temp.next
        print("Value not found.")

    def delete_beginning(self):
        if self.head is None:
            print("List is empty.")
            return
        deleted_data = self.head.data
        self.head = self.head.next
        print(f"Deleted {deleted_data} from the beginning.")

    def delete_end(self):
        if self.head is None:
            print("List is empty.")
            return
        if self.head.next is None:
            deleted_data = self.head.data
            self.head = None
            print(f"Deleted {deleted_data} from the end.")
            return
        temp = self.head
        while temp.next.next is not None:
            temp = temp.next
        deleted_data = temp.next.data
        temp.next = None
        print(f"Deleted {deleted_data} from the end.")

    def count_nodes(self):
        count = 0
        temp = self.head
        while temp is not None:
            count += 1
            temp = temp.next
        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

sll = SLL()
```

```
while True:
    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()
    elif choice == 2:
        sll.insert_beginning()
    elif choice == 3:
        sll.insert_end()
    elif choice == 4:
        sll.insert_at_index()
    elif choice == 5:
        sll.delete_by_value()
    elif choice == 6:
        sll.delete_beginning()
    elif choice == 7:
        sll.delete_end()
    elif choice == 8:
        sll.count_nodes()
    elif choice == 9:
        sll.display()
    elif choice == 10:
        print("Exiting from the program...")
        break
    else:
        print("Invalid choice.”)
```

OUTPUT:

```
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 4/1.py ========

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 2
Enter data: 6
Enter data: 7
Enter data: 5
Linked List created successfully.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Exit
Enter your choice: 10
Exiting from the program...
```

EXPLANATION:

- Defines Node and SLL classes to create and manage a singly linked list.
- The create function creates nodes and connects them using the next pointer.
- Provides operations to insert nodes at the beginning, end, and a specific index.
- Provides operations to delete a node by value, from the beginning, and from the end.
- The count and display functions show the number of nodes and the contents of the list.
- A menu driven loop allows the user to select and perform different linked list operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	IndentationError in class or function definitions	Maintain proper indentation inside classes and functions
2	AttributeError while accessing next	Check that the node is created correctly before accessing next
3	Index out of range during insertion	Enter an index within the valid linked list range

2. AIM:

Write a python program to implement a Double linked list which could perform the below operations ,by taking an input from the user.

1)Insert at beginning
2)Insert at end
3)insert at index
4)Delete by value
5)Delete First Node
6)Delete Last Node
7)Display the Forward Nodes
8)Display the Backward Nodes
9)Exit

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.prev = None
        self.data = data
        self.next = None

class DoublyLinkedList:
    def __init__(self):
```

```
    self.head = None

def insert_begin(self, data):
    new = Node(data)
    if self.head is None:
        self.head = new
    else:
        new.next = self.head
        self.head.prev = new
        self.head = new
    print("Insertion Completed")

def insert_end(self, data):
    new = Node(data)
    if self.head is None:
        self.head = new
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new
        new.prev = temp
    print("Insertion Completed")

def insert_index(self, index, data):
    if index < 0:
        print("Invalid Index")
    elif index == 0:
        self.insert_begin(data)
    else:
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Invalid Index")
                return
            temp = temp.next
        if temp is None:
            print("Invalid Index")
            return
        new = Node(data)
        new.next = temp.next
        new.prev = temp
        if temp.next:
            temp.next.prev = new
        temp.next = new
        print("Insertion Completed")

def deleteBeg(self):
    if self.head is None:
        print("Can't perform delete operation")
    elif self.head.next is None:
        self.head = None
        print("Value Deleted")
    else:
        self.head.next.prev = None
        self.head = self.head.next
        print("Value Deleted")

def deleteEnd(self):
    if self.head is None:
```

```
            print("Can't perform delete operation")
        elif self.head.next is None:
            self.head = None
            print("Value Deleted")
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.prev.next = None
            print("Value Deleted")

    def deleteIndex(self, index):
        if self.head is None:
            print("Can't perform delete operation")
        elif index < 0:
            print("Invalid Index")
        elif index == 0:
            self.deleteBeg()
        else:
            temp = self.head
            for i in range(index):
                if temp is None:
                    print("Invalid Index")
                    return
                temp = temp.next
            if temp is None:
                print("Invalid Index")
            elif temp.next:
                temp.next.prev = temp.prev
                temp.prev.next = temp.next
                print("Value Deleted")
            else:
                temp.prev.next = None
                print("Value Deleted")

    def displayFwd(self):
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")

    def displayBwd(self):
        if self.head is None:
            print("None")
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.prev
        print("None")

Dll = DoublyLinkedList()

while True:
    print("\n1.Insert Beginning")
    print("2.Insert End")
    print("3.Insert at Index")
```

```
print("4.Delete at Beginning")
print("5.Delete at End")
print("6.Delete at Specific Index")
print("7.Display Forward")
print("8.Display Backward")
print("9.Exit")

ch = int(input("Choice: "))

if ch == 1:
    x = int(input("Value: "))
    Dll.insert_begin(x)
    Dll.displayFwd()
elif ch == 2:
    x = int(input("Value: "))
    Dll.insert_end(x)
    Dll.displayFwd()
elif ch == 3:
    idx = int(input("Index: "))
    x = int(input("Value: "))
    Dll.insert_index(idx, x)
    Dll.displayFwd()
elif ch == 4:
    Dll.deleteBeg()
    Dll.displayFwd()
elif ch == 5:
    Dll.deleteEnd()
    Dll.displayFwd()
elif ch == 6:
    idx = int(input("Index: "))
    Dll.deleteIndex(idx)
    Dll.displayFwd()
elif ch == 7:
    Dll.displayFwd()
elif ch == 8:
    Dll.displayBwd()
elif ch == 9:
    print("Loop Terminated")
    break
else:
    print("Invalid Choice")
```

OUTPUT:

```
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 4/2.py ========

1.Insert Beginning
2.Insert End
3.Insert at Index
4.Delete at Beginning
5.Delete at End
6.Delete at Specific Index
7.Display Forward
8.Display Backward
9.Exit
Choice: 1
Value: 4
Insertion Completed
4 <-> None

1.Insert Beginning
2.Insert End
3.Insert at Index
4.Delete at Beginning
5.Delete at End
6.Delete at Specific Index
7.Display Forward
8.Display Backward
9.Exit
Choice: 6
Index: 1
Invalid Index
4 <-> None

1.Insert Beginning
2.Insert End
3.Insert at Index
4.Delete at Beginning
5.Delete at End
6.Delete at Specific Index
7.Display Forward
8.Display Backward
9.Exit
Choice: 9
Loop Terminated
```

EXPLANATION:

- Defines a Node class with data, previous link and next link for a doubly linked list.
- Provides functions to insert nodes at the beginning, end and a specific index.
- Provides functions to delete nodes from the beginning, end and a specific index.
- Maintains both prev and next links while inserting and deleting nodes.
- The display functions show the list in forward and backward directions.
- A menu allows the user to perform different doubly linked list operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Invalid Index during insertion	Enter an index within the valid list range
2	Links are not updated correctly after deletion	Update both prev and next links properly
3	Backward display gives incorrect order	Traverse to the last node before moving backward

3. AIM:

Write a python program to implement a Circular linked list which could perform the below operations ,by taking an input from the user.

1)create a linked list
2)Insert at beginning
3)Insert at end
4)insert at index
5)Delete by value
6)Delete First Node
7)Delete Last Node
8)Count number of nodes
9)Display
a)Head to Tail
b)Tail to Head
10)Exit

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))
        self.head = None
        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)
```

```
            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head
                while temp.next != self.head:
                    temp = temp.next
                temp.next = new_node
                new_node.next = self.head
        print("Circular Linked List created successfully.")

    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            new_node.next = self.head
            temp.next = new_node
            self.head = new_node
        print(f"Inserted {data} at the beginning.")

    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head
        print(f"Inserted {data} at the end.")

    def insert_at_index(self):
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))
        if index < 0:
            print("Invalid index.")
            return
        if index == 0:
            self.insert_beginning()
            return
        if self.head is None:
            print("Index out of range.")
            return
        temp = self.head
        for i in range(index - 1):
            temp = temp.next
            if temp == self.head:
                print("Index out of range.")
                return
        new_node = Node(data)
        new_node.next = temp.next
```

```
        temp.next = new_node
        print(f"Inserted {data} at index {index}.")

    def delete_by_value(self):
        value = int(input("Enter value to delete: "))
        if self.head is None:
            print("Circular Linked List is empty.")
            return
        if self.head.data == value:
            if self.head.next == self.head:
                self.head = None
            else:
                temp = self.head
                while temp.next != self.head:
                    temp = temp.next
                temp.next = self.head.next
                self.head = self.head.next
            print(f"Deleted {value}.")
            return
        temp = self.head
        while temp.next != self.head:
            if temp.next.data == value:
                temp.next = temp.next.next
                print(f"Deleted {value}.")
                return
            temp = temp.next
        print(f"{value} not found in the Linked List.")

    def delete_first(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return
        deleted = self.head.data
        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = self.head.next
            self.head = self.head.next
        print(f"Deleted first node: {deleted}")

    def delete_last(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return
        if self.head.next == self.head:
            deleted = self.head.data
            self.head = None
            print(f"Deleted last node: {deleted}")
            return
        temp = self.head
        while temp.next.next != self.head:
            temp = temp.next
        deleted = temp.next.data
        temp.next = self.head
        print(f"Deleted last node: {deleted}")

    def count_nodes(self):
```

```
        if self.head is None:
            print("Number of nodes: 0")
            return
        count = 0
        temp = self.head
        while True:
            count += 1
            temp = temp.next
            if temp == self.head:
                break
        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return
        temp = self.head
        print("Circular Linked List:", end=" ")
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("HEAD")

    def display_head_tail(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        print("Head:", self.head.data)
        print("Tail:", temp.data)

    def display_tail_to_head(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return
        tail = self.head
        while tail.next != self.head:
            tail = tail.next
        print("Data from Tail to Head:", end=" ")
        current = tail
        while True:
            print(current.data, end=" ")
            if current == self.head:
                break
            temp = self.head
            while temp.next != current:
                temp = temp.next
            current = temp
        print()

cll = CircularLinkedList()

while True:
    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
```

```
print("3. Insert at End")
print("4. Insert at Specific Index")
print("5. Delete by Value")
print("6. Delete First Node")
print("7. Delete Last Node")
print("8. Count Number of Nodes")
print("9. Display")
print("10. Display Head and Tail")
print("11. Print Data from Tail to Head")
print("12. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    cll.create()
elif choice == 2:
    cll.insert_beginning()
elif choice == 3:
    cll.insert_end()
elif choice == 4:
    cll.insert_at_index()
elif choice == 5:
    cll.delete_by_value()
elif choice == 6:
    cll.delete_first()
elif choice == 7:
    cll.delete_last()
elif choice == 8:
    cll.count_nodes()
elif choice == 9:
    cll.display()
elif choice == 10:
    cll.display_head_tail()
elif choice == 11:
    cll.display_tail_to_head()
elif choice == 12:
    print("Exiting program....")
    break
else:
    print("Invalid choice. Please try again.")
```

OUTPUT:

```
================================ RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 4/3.py ================================

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 2
Enter data: 7
Enter data: 9
Circular Linked List created successfully.

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display
10. Display Head and Tail
11. Print Data from Tail to Head
12. Exit
Enter your choice: 9
Circular Linked List: 2 -> 7 -> 9 -> HEAD
```

EXPLANATION:

- Defines Node and CircularLinkedList classes where the last node points back to the head.
- Provides functions to create and insert nodes at the beginning, end and a specific index.
- Provides functions to delete nodes by value, from the beginning and from the end.
- Count and display functions traverse the list until they reach the head again.
- Additional functions display the head and tail and traverse the list from tail to head.
- A menu allows the user to perform different circular linked list operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Infinite loop while traversing the list	Stop traversal when the current node reaches the head
2	Incorrect head after deletion	Update the head and last node link correctly
3	Index out of range during insertion	Enter an index within the valid list range

LAB 5

1. AIM:

Write a python program to implement a stack using an array, also implement the stack operations such as:
a)push
b)pop
c)peek also display the stack elements

PROGRAM:

```
class StackEx:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            for i in range(self.top, -1, -1):
                print(self.stack[i])
```

```
size = int(input("Enter the size of the stack: "))
s = StackEx(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 5/1.py ========
Enter the size of the stack: 3

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 7
7 pushed into the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 18
18 pushed into the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 18

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

EXPLANATION:

- Defines a StackEx class that creates a stack with a fixed size and initializes top as -1.
- The push function adds an element to the top and checks for stack overflow.
- The pop function removes the top element and checks for stack underflow.
- The peek function displays the current top element without removing it.
- The display function prints all stack elements from top to bottom.
- A menu allows the user to perform push, pop, peek, display and exit operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Stack Overflow occurs when adding elements	Increase the stack size or remove existing elements
2	Stack Underflow occurs while popping	Check whether the stack contains elements before popping
3	IndentationError in class or menu section	Maintain proper indentation throughout the program

2. AIM:

Write a python program to implement a stack using linked list ,also implement the stack operations such as:

a)push

b)pop

c)peek also display the stack elements

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "pushed into stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            print(temp.data, "popped from stack")
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next
```

```
s = Stack()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.push(data)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 5/1.py ========
Enter the size of the stack: 3

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 7
7 pushed into the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 18
18 pushed into the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the stack are:
18
7
```

EXPLANATION:

- Defines a Node class to store data and the link to the next node.
- The Stack class uses top to keep track of the top node.
- The push function creates a new node and adds it to the top of the stack.
- The pop function removes the top node and handles stack underflow.
- The peek and display functions show the top element and all stack elements.
- A menu allows the user to perform different stack operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Stack Underflow while popping	Check whether the stack is empty before popping
2	AttributeError while accessing node data	Check that top points to a valid node
3	IndentationError in class or menu section	Maintain proper indentation throughout the program

LAB 6

1. AIM:

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach.Implement using array data structures.

PROGRAM:

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter queue size: "))
q = Queue(size)

while True:
```

```
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        x = int(input("Enter element: "))
        q.enqueue(x)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program terminated")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 6/1.py ========
Enter queue size: 3
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element: 3
3 inserted into the queue
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 3
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

EXPLANATION:

- Defines a Queue class using a fixed size list and initializes front and rear as -1.
- The enqueue function adds elements at the rear and checks for queue overflow.
- The dequeue function removes elements from the front and checks for queue underflow.
- The peek function displays the front element without removing it.
- The display function prints all elements from front to rear.
- A menu allows the user to perform enqueue, dequeue, peek, display and exit operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Queue Overflow when inserting an element	Increase the queue size or remove existing elements
2	Queue Underflow while deleting an element	Check whether the queue contains elements before deletion
3	Elements are not displayed correctly	Check the front and rear values before displaying

2. AIM:

Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach.Implement using linked list data structures.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class QueueEx:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, x):
        new = Node(x)

        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new

        print(x, "inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
```

```
        else:
            x = self.front.data
            self.front = self.front.next

            if self.front is None:
                self.rear = None

            print(x, "deleted from the queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")

            temp = self.front
            while temp is not None:
                print(temp.data)
                temp = temp.next


q = QueueEx()

while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice”)
```

OUTPUT:

```
================================ RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 6/2.py ================================

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 7
7 inserted into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 18
18 inserted into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
7
18
```

EXPLANATION:

Defines a Node class to store the queue element and the link to the next node.
QueueEx uses front and rear pointers to manage the queue.
The enqueue function adds a new node at the rear of the queue.
The dequeue function removes a node from the front and handles an empty queue.
The peek and display functions show the front element and all queue elements.
A menu allows the user to perform enqueue, dequeue, peek, display and exit operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Queue Underflow during dequeue	Check whether front is None before deleting
2	Rear pointer becomes incorrect	Set rear to None when the last node is removed
3	Elements are not displayed correctly	Check the next link while traversing the queue

3. AIM:

Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu-driven approach.

PROGRAM:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into the queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = (self.front + 1) % self.size
            print(item, "deleted from the queue")

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while True:
                print(self.queue[i])
                if i == self.rear:
                    break
                i = (i + 1) % self.size


size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)

while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.display()
    elif choice == 4:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
================================ RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 6/3.py ================================
Enter the size of the queue: 4

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 3
3 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 6
6 inserted into the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
3 deleted from the queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Program terminated.
```

EXPLANATION:

- Defines a CircularQueue class using a fixed size list with front and rear pointers.
- The enqueue function inserts elements and uses modulo to move rear circularly.
- The dequeue function removes elements from the front and updates the front position.
- The queue checks for overflow when the next rear position reaches the front.
- The display function starts from front and moves circularly until it reaches rear.
- A menu allows the user to perform enqueue, dequeue, display and exit operations.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Queue Overflow occurs unexpectedly	Check the circular full condition using front and rear
2	Queue Underflow while deleting	Check whether front is set to -1 before deletion
3	Incorrect elements are displayed	Check the circular movement using modulo operation

LAB 7

1. AIM:

Write a python program to implement Binary tree and its operations such as insert, delete, search, all 3 traversals using arrays.

PROGRAM:

```
class BinaryTreeArray:
    def __init__(self,size):
        self.tree=[None]*size
        self.size=size

    def set_root(self,data):
        self.tree[0]=data

    def set_left(self,parent_index,data):
        child_index=2*parent_index+1
        if child_index<self.size:
            self.tree[child_index]=data
        else:
            print("Index out of range!")

    def set_right(self,parent_index,data):
        child_index=2*parent_index+2
        if child_index<self.size:
            self.tree[child_index]=data
        else:
            print("Index out of range!")

    def preorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        print(self.tree[index],end=" ")
        self.preorder(2*index+1)
        self.preorder(2*index+2)

    def inorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        self.inorder(2*index+1)
        print(self.tree[index],end=" ")
        self.inorder(2*index+2)

    def postorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        self.postorder(2*index+1)
        self.postorder(2*index+2)
        print(self.tree[index],end=" ")

    def level_order(self):
        for value in self.tree:
            if value is not None:
                print(value,end=" ")
```

```
    def display(self):
        print("\nArray Representation:")
        for i in range(self.size):
            print(f"Index {i}: {self.tree[i]}")

tree=BinaryTreeArray(7)
tree.set_root('A')
tree.set_left(0,'B')
tree.set_right(0,'C')
tree.set_left(1,'D')
tree.set_right(1,'E')
tree.set_right(2,'F')
tree.display()
print("\nPreorder: ",end="")
tree.preorder()
print("\nInorder: ",end="")
tree.inorder()
print("\nPostorder: ",end="")
tree.postorder()
print("\nLevel-order: ",end="")
tree.level_order()
```

OUTPUT:

```
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 7/7a.py =======

Array Representation:
Index 0: A
Index 1: B
Index 2: C
Index 3: D
Index 4: E
Index 5: None
Index 6: F

Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
```

EXPLANATION:

- Defines a BinaryTreeArray class to store binary tree nodes in an array of a fixed size.
- The set_root function assigns the root node, while set_left and set_right add left and right child nodes using array indexes.
- The preorder function visits the root first, followed by the left and right subtrees.
- The inorder function visits the left subtree, root and right subtree in order.
- The postorder function visits the left and right subtrees before the root, while level_order displays nodes level by level.
- The display function prints the array representation of the tree, and the main program creates a tree and displays all traversal results.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Index out of range when adding a child to a parent near the end of the array	Check whether the child index is within the array size before inserting the node
2	Traversal gives incomplete results when a parent node is empty	Ensure the parent node is assigned before adding its child nodes
3	Program gives an index error when the tree size is set to zero	Create the tree with a positive size before assigning the root node

2. AIM:

Write a python program to implement Binary tree and its operations such as insert, delete, search, all 3 traversals using linked list.

PROGRAM:

```
class Node:
    def __init__(self,data):
        self.data=data
        self.left=None
        self.right=None

class BinaryTree:
    def __init__(self):
        self.root=None

    def create_tree(self):
        self.root=Node('A')
        self.root.left=Node('B')
        self.root.right=Node('C')
        self.root.left.left=Node('D')
        self.root.left.right=Node('E')
        self.root.right.right=Node('F')

    def preorder(self,node):
        if node is None:
            return
        print(node.data,end=" ")
        self.preorder(node.left)
        self.preorder(node.right)

    def inorder(self,node):
        if node is None:
            return
        self.inorder(node.left)
        print(node.data,end=" ")
        self.inorder(node.right)

    def postorder(self,node):
        if node is None:
            return
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data,end=" ")

    def level_order(self):
        if self.root is None:
            return
```

```
        queue=[self.root]
        while queue:
            node=queue.pop(0)
            print(node.data,end=" ")
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)

tree=BinaryTree()
tree.create_tree()

print("Preorder: ",end="")
tree.preorder(tree.root)

print("\nInorder: ",end="")
tree.inorder(tree.root)

print("\nPostorder: ",end="")
tree.postorder(tree.root)

print("\nLevel-order: ",end="")
tree.level_order()
```

OUTPUT:

```
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 7/7b.py =======
Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
```

EXPLANATION:

- Defines a Node class to store data and references to the left and right child nodes.
- Defines a BinaryTree class that initializes an empty tree with no root node.
- The create_tree function creates a binary tree with A as the root and adds the remaining nodes as children.
- The preorder function visits the root first, followed by the left and right subtrees.
- The inorder and postorder functions visit nodes in their respective orders, while level_order uses a queue to visit nodes level by level.

ERROR TABLE:

S.No.	Error Obtained	Rectification
1	Attribute error when accessing a child of a node that was not created	Create the required node before assigning its children
2	Traversal produces no output when the root is not initialized	Call the create_tree function before performing traversal
3	Incorrect traversal output due to calling the traversal function with the wrong starting node	Pass the root node as the starting point for preorder, inorder and postorder

LAB 8

1. AIM:

Write a python program to implement Binary search tree(BST) using arrays.

PROGRAM:

```
tree = []

def insert(value, i=0):
    global tree

    if i >= len(tree):
        tree.extend([None] * (i - len(tree) + 1))

    if tree[i] is None:
        tree[i] = value
    elif value < tree[i]:
        insert(value, 2 * i + 1)
    elif value > tree[i]:
        insert(value, 2 * i + 2)
    else:
        print("Duplicate values are not allowed.")


def search(value, i=0):
    if i >= len(tree) or tree[i] is None:
        return False

    if tree[i] == value:
        return True
    elif value < tree[i]:
        return search(value, 2 * i + 1)
    else:
        return search(value, 2 * i + 2)


def minimum(i):
    while 2 * i + 1 < len(tree) and tree[2 * i + 1] is not None:
        i = 2 * i + 1
    return i


def delete(value, i=0):
    if i >= len(tree) or tree[i] is None:
        return

    if value < tree[i]:
        delete(value, 2 * i + 1)
    elif value > tree[i]:
        delete(value, 2 * i + 2)
    else:
        left = 2 * i + 1
        right = 2 * i + 2

        has_left = left < len(tree) and tree[left] is not None
        has_right = right < len(tree) and tree[right] is not None
```

```
        if not has_left and not has_right:
            tree[i] = None

        elif not has_left:
            tree[i] = tree[right]
            delete_subtree(right, i)

        elif not has_right:
            tree[i] = tree[left]
            delete_subtree(left, i)

        else:
            j = minimum(right)
            tree[i] = tree[j]
            delete(tree[j], right)


def delete_subtree(child, parent):
    if child >= len(tree) or tree[child] is None:
        return

    value = tree[child]
    delete(value, child)

    remaining = collect_subtree(child)
    for v in remaining:
        insert(v)


def collect_subtree(i):
    if i >= len(tree) or tree[i] is None:
        return []

    value = tree[i]
    left = collect_subtree(2 * i + 1)
    right = collect_subtree(2 * i + 2)
    tree[i] = None

    return left + [value] + right


def inorder(i=0):
    if i < len(tree) and tree[i] is not None:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)


def preorder(i=0):
    if i < len(tree) and tree[i] is not None:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)


def postorder(i=0):
    if i < len(tree) and tree[i] is not None:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
```

```
        print(tree[i], end=" ")


while True:
    print("\n\n1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display Array")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        insert(value)

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if search(value):
            delete(value)
            print("Value deleted.")
        else:
            print("Value not found.")

    elif choice == 3:
        value = int(input("Enter value to search: "))
        print("Found" if search(value) else "Not found")

    elif choice == 4:
        inorder()
        print()

    elif choice == 5:
        preorder()
        print()

    elif choice == 6:
        postorder()
        print()

    elif choice == 7:
        print(tree)

    elif choice == 8:
        break

    else:
        print("Invalid choice.”)
```

OUTPUT:

```
>>>
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 8/8a.py =======


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Array
8. Exit
Enter your choice: 1
Enter value: 18


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Array
8. Exit
Enter your choice: 8
```

EXPLANATION:

- Creates a binary search tree using an array, where smaller values are stored in the left subtree and larger values in the right subtree.
- The insert function adds new values at the correct position and prevents duplicate values.
- The search function checks whether a given value exists, while the minimum function finds the smallest value in a subtree.
- The delete function removes a node based on whether it has no children, one child or two children.
- The inorder, preorder and postorder functions display the tree elements in different traversal orders.
- A menu allows the user to insert, delete and search values, perform traversals, display the array and exit the program.

ERROR TABLE:

S. No.	Error Obtained	Rectification
1	Name error when delete_subtree or collect_subtree is not defined	Define both functions before calling them
2	Incorrect tree structure after deleting a node with one child	Reorganize the remaining subtree carefully after deletion
3	Type error when entering text instead of a number in the menu	Enter a valid integer for the menu choice and tree value

2. AIM:

Write a python program to implement Binary search tree(BST) using linked list.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def insert(root, value):
    if root is None:
        return Node(value)

    if value < root.data:
        root.left = insert(root.left, value)
    elif value > root.data:
        root.right = insert(root.right, value)
    else:
        print("Duplicate values are not allowed.")

    return root


def search(root, value):
    if root is None:
        return False

    if root.data == value:
        return True
    elif value < root.data:
        return search(root.left, value)
    else:
        return search(root.right, value)


def find_min(root):
    while root.left is not None:
        root = root.left
    return root


def delete(root, value):
    if root is None:
        return None

    if value < root.data:
        root.left = delete(root.left, value)
    elif value > root.data:
        root.right = delete(root.right, value)
    else:
        if root.left is None:
            return root.right

        if root.right is None:
            return root.left
```

```
        temp = find_min(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root


def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")


root = None

while True:
    print("\n\n1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        root = insert(root, value)

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if search(root, value):
            root = delete(root, value)
            print("Value deleted.")
        else:
            print("Value not found.")

    elif choice == 3:
        value = int(input("Enter value to search: "))
        if search(root, value):
            print("Value found.")
        else:
            print("Value not found.")
```

```
    elif choice == 4:
        inorder(root)
        print()

    elif choice == 5:
        preorder(root)
        print()

    elif choice == 6:
        postorder(root)
        print()

    elif choice == 7:
        break

    else:
        print("Invalid choice.”)
```

OUTPUT:

```
>>>
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 8/8a.py =======


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Array
8. Exit
Enter your choice: 1
Enter value: 37


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Array
8. Exit
Enter your choice: 7
[37]


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Array
8. Exit
Enter your choice: 8
```

EXPLANATION:

- Defines a Node class to store data and references to the left and right child nodes.
- The insert function adds values to the correct position in the binary search tree and prevents duplicate values.
- The search function checks whether a value exists, while find_min finds the smallest value in a subtree.
- The delete function removes a node with no children, one child or two children while maintaining the tree structure.
- The inorder, preorder and postorder functions display the tree elements in different traversal orders.
- A menu allows the user to insert, delete and search values, perform traversals and exit the program.

ERROR TABLE:

S.N	Error Obtained	Rectification
1	Root node remains unchanged after inserting the first value	Assign the returned node to root using root equals insert of root and value
2	Tree structure becomes incorrect if the returned child node is not assigned during deletion	Update the left or right child with the result of the delete function
3	Invalid menu input causes the program to stop	Enter a valid integer for the menu choice

LAB 9

1. AIM:

Write a python program to implement AVL tree using arrays.

PROGRAM:

```
keys = []
left = []
right = []
heights = []
root = -1


def new_node(key):
    i = len(keys)
    keys.append(key)
    left.append(-1)
    right.append(-1)
    heights.append(1)
    return i


def height(i):
    return 0 if i == -1 else heights[i]


def update(i):
    heights[i] = 1 + max(height(left[i]), height(right[i]))


def balance(i):
    return 0 if i == -1 else height(left[i]) - height(right[i])


def rotate_right(y):
    x = left[y]
    t = right[x]

    right[x] = y
    left[y] = t

    update(y)
    update(x)
    return x


def rotate_left(x):
    y = right[x]
    t = left[y]

    left[y] = x
    right[x] = t

    update(x)
    update(y)
    return y
```

```
def insert(node, key):
    if node == -1:
        return new_node(key)

    if key < keys[node]:
        left[node] = insert(left[node], key)
    elif key > keys[node]:
        right[node] = insert(right[node], key)
    else:
        return node

    update(node)
    bf = balance(node)

    if bf > 1 and key < keys[left[node]]:
        return rotate_right(node)

    if bf < -1 and key > keys[right[node]]:
        return rotate_left(node)

    if bf > 1 and key > keys[left[node]]:
        left[node] = rotate_left(left[node])
        return rotate_right(node)

    if bf < -1 and key < keys[right[node]]:
        right[node] = rotate_right(right[node])
        return rotate_left(node)

    return node


def min_node(node):
    while left[node] != -1:
        node = left[node]
    return node


def delete(node, key):
    if node == -1:
        return -1

    if key < keys[node]:
        left[node] = delete(left[node], key)
    elif key > keys[node]:
        right[node] = delete(right[node], key)
    else:
        if left[node] == -1:
            return right[node]

        if right[node] == -1:
            return left[node]

        temp = min_node(right[node])
        keys[node] = keys[temp]
        right[node] = delete(right[node], keys[temp])

    update(node)
    bf = balance(node)
```

```
    if bf > 1 and balance(left[node]) >= 0:
        return rotate_right(node)

    if bf > 1 and balance(left[node]) < 0:
        left[node] = rotate_left(left[node])
        return rotate_right(node)

    if bf < -1 and balance(right[node]) <= 0:
        return rotate_left(node)

    if bf < -1 and balance(right[node]) > 0:
        right[node] = rotate_right(right[node])
        return rotate_left(node)

    return node


def search(node, key):
    while node != -1:
        if keys[node] == key:
            return True

        if key < keys[node]:
            node = left[node]
        else:
            node = right[node]

    return False


def inorder(node):
    if node != -1:
        inorder(left[node])
        print(keys[node], end=" ")
        inorder(right[node])


def preorder(node):
    if node != -1:
        print(keys[node], end=" ")
        preorder(left[node])
        preorder(right[node])


def postorder(node):
    if node != -1:
        postorder(left[node])
        postorder(right[node])
        print(keys[node], end=" ")


while True:
    print("\n\n--- AVL TREE USING ARRAYS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
```

```
print("7. Exit")

choice = input("Enter choice: ")

if choice == '1':
    key = int(input("Enter value: "))
    root = insert(root, key)
    print("Node inserted.")

elif choice == '2':
    key = int(input("Enter value: "))
    if search(root, key):
        root = delete(root, key)
        print("Node deleted.")
    else:
        print("Value not found.")

elif choice == '3':
    key = int(input("Enter value: "))
    print("Value found." if search(root, key)
          else "Value not found.")

elif choice == '4':
    inorder(root)
    print()

elif choice == '5':
    preorder(root)
    print()

elif choice == '6':
    postorder(root)
    print()

elif choice == '7':
    break

else:
    print("Invalid choice.”)
```

OUTPUT:

```
>>>
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 9/9a.py =======


--- AVL TREE USING ARRAYS ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 1
Enter value: 7
Node inserted.


--- AVL TREE USING ARRAYS ---
1. Insert
...
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 2
Enter value: 7
Node deleted.


--- AVL TREE USING ARRAYS ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 7
```

EXPLANATION:

- Uses arrays to store node values, left and right child indexes, and node heights for an AVL tree.
- The new_node function creates a node, while height, update and balance calculate and maintain tree heights and balance factors.
- The insert function adds values and performs rotations when the tree becomes unbalanced.
- The delete function removes nodes and rebalances the tree using left and right rotations.
- The search function finds a value, while inorder, preorder and postorder display nodes in different traversal orders.
- A menu allows the user to insert, delete and search values, perform traversals and exit the program.

ERROR TABLE:

S. No	Error Obtained	Rectification
1	Incorrect height values after inserting or deleting nodes	Update the height of each affected node after modifying its children
2	Index error when accessing a child during rotation	Check that the required child exists before performing the rotation
3	AVL tree becomes unbalanced after deletion	Calculate the balance factor and apply the appropriate rotation after deletion

2. AIM:
Write a python program to implement AVL tree using linked list.

PROGRAM:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1


def height(root):
    return 0 if root is None else root.height


def balance(root):
    if root is None:
        return 0
    return height(root.left) - height(root.right)


def update(root):
    root.height = 1 + max(height(root.left), height(root.right))


def rotate_right(y):
    x = y.left
    t = x.right

    x.right = y
    y.left = t

    update(y)
    update(x)
    return x


def rotate_left(x):
    y = x.right
    t = y.left

    y.left = x
    x.right = t

    update(x)
    update(y)
```

```
    return y


def insert(root, key):
    if root is None:
        return Node(key)

    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    else:
        return root

    update(root)
    bf = balance(root)

    if bf > 1 and key < root.left.key:
        return rotate_right(root)

    if bf < -1 and key > root.right.key:
        return rotate_left(root)

    if bf > 1 and key > root.left.key:
        root.left = rotate_left(root.left)
        return rotate_right(root)

    if bf < -1 and key < root.right.key:
        root.right = rotate_right(root.right)
        return rotate_left(root)

    return root


def min_node(root):
    while root.left is not None:
        root = root.left
    return root


def delete(root, key):
    if root is None:
        return None

    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = min_node(root.right)
        root.key = temp.key
        root.right = delete(root.right, temp.key)

    update(root)
    bf = balance(root)
```

```
    if bf > 1 and balance(root.left) >= 0:
        return rotate_right(root)

    if bf > 1 and balance(root.left) < 0:
        root.left = rotate_left(root.left)
        return rotate_right(root)

    if bf < -1 and balance(root.right) <= 0:
        return rotate_left(root)

    if bf < -1 and balance(root.right) > 0:
        root.right = rotate_right(root.right)
        return rotate_left(root)

    return root


def search(root, key):
    if root is None or root.key == key:
        return root

    if key < root.key:
        return search(root.left, key)

    return search(root.right, key)


def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)


def preorder(root):
    if root:
        print(root.key, end=" ")
        preorder(root.left)
        preorder(root.right)


def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.key, end=" ")


root = None

while True:
    print("\n\n--- AVL TREE USING LINKED LIST ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Exit")
```

```
choice = input("Enter choice: ")

if choice == '1':
    key = int(input("Enter value: "))
    root = insert(root, key)
    print("Node inserted.")

elif choice == '2':
    key = int(input("Enter value: "))
    if search(root, key):
        root = delete(root, key)
        print("Node deleted.")
    else:
        print("Value not found.")

elif choice == '3':
    key = int(input("Enter value: "))
    if search(root, key):
        print("Value found.")
    else:
        print("Value not found.")

elif choice == '4':
    inorder(root)
    print()

elif choice == '5':
    preorder(root)
    print()

elif choice == '6':
    postorder(root)
    print()

elif choice == '7':
    print("Exiting program.")
    break

else:
    print("Invalid choice.”)
```

OUTPUT:

```
>>>
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 9/9b.py =======


--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 1
Enter value: 5
Node inserted.


--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 3
Enter value: 5
Value found.


--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 7
Exiting program.
```

EXPLANATION:

- Defines a Node class to store the key, left and right child references, and height of each node.
- The height, balance and update functions calculate node heights and check whether the AVL tree is balanced.
- The insert function adds values in binary search tree order and performs rotations to maintain balance.
- The delete function removes nodes and rebalances the tree using suitable rotations.
- The search function finds a key, while inorder, preorder and postorder display the nodes in different traversal orders.
- A menu allows the user to insert, delete and search values, perform traversals and exit the program.

ERROR TABLE

S.No.	Error Obtained	Rectification
1	Attribute error when accessing the height of a missing node	Check whether the node is None before accessing its height
2	Incorrect tree balance after insertion	Update node heights and apply the appropriate rotation
3	Root node is lost after deletion or rotation	Assign the returned node to root after calling insert or delete

LAB 10

1. AIM:

Write a python program to implement hashing using separate chaining.

PROGRAM:

```
size = 10

def chaining(keys):
    table = [[] for i in range(size)]

    for key in keys:
        table[key % size].append(key)

    print("Separate Chaining:")
    for i in range(size):
        print(i, ":", table[i])

def linear(keys):
    table = [-1] * size

    for key in keys:
        index = key % size
        count = 0

        while table[index] != -1 and count < size:
            index = (index + 1) % size
            count += 1

        if count < size:
            table[index] = key
        else:
            print("Table full:", key)

    print("Linear Probing:")
    for i in range(size):
        print(i, ":", table[i])

def quadratic(keys):
    table = [-1] * size

    for key in keys:
        index = key % size
        inserted = False

        for i in range(size):
            pos = (index + i*i) % size

            if table[pos] == -1:
                table[pos] = key
                inserted = True
                break

        if not inserted:
            print("Could not insert:", key)
```

```
    print("Quadratic Probing:")
    for i in range(size):
        print(i, ":", table[i])

n = int(input("Enter number of elements: "))
keys = []

for i in range(n):
    keys.append(int(input("Enter key: ")))

print("1. Separate Chaining")
print("2. Linear Probing")
print("3. Quadratic Probing")

ch = int(input("Enter choice: "))

if ch == 1:
    chaining(keys)
elif ch == 2:
    linear(keys)
elif ch == 3:
    quadratic(keys)
else:
    print("Invalid choice")
```

OUTPUT:

```
>>>
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 10/a.py =======
Enter number of elements: 5
Enter key: 7
Enter key: 13
Enter key: 2
Enter key: 18
Enter key: 5
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
Enter choice: 1
Separate Chaining:
0 : []
1 : []
2 : [2]
3 : [13]
4 : []
5 : [5]
6 : []
7 : [7]
8 : [18]
9 : []
```

EXPLANATION:

- Defines three hashing methods using arrays, namely separate chaining, linear probing and quadratic probing.
- The chaining function creates a list of buckets and stores keys with the same hash index in the same bucket.
- The linear function resolves collisions by checking the next available position sequentially using linear probing.
- The quadratic function resolves collisions by checking positions using the square of the probing number.
- Both probing methods check whether the hash table is full before inserting a key and display a message if insertion fails.
- The main program accepts keys from the user, displays a menu and executes the selected hashing method.

ERROR TABLE:

S. No	Error Obtained	Rectification
1	Table full message appears when all positions are occupied during linear probing	Check the table capacity before inserting additional keys
2	Quadratic probing fails to find an empty position even when some slots are available	Use a suitable table size and probing method
3	Invalid hashing method is selected from the menu	Enter a valid choice between 1 and 3

2. AIM:

Write a python program to implement hashing using linear probing.

PROGRAM:

```
size = 10

class Node:
    def __init__(self, key=-1):
        self.key = key
        self.next = None

def create_table():
    head = Node()
    temp = head
    for i in range(size-1):
        temp.next = Node()
        temp = temp.next
    return head

def get_node(head, index):
    temp = head
```

```
    for i in range(index):
        temp = temp.next
    return temp

def chaining(keys):
    table = [None]*size
    for key in keys:
        index = key%size
        newnode = Node(key)
        newnode.next = table[index]
        table[index] = newnode
    print("Separate Chaining:")
    for i in range(size):
        print(i,":",end=" ")
        temp = table[i]
        while temp:
            print(temp.key,end=" -> ")
            temp = temp.next
        print("None")

def linear(keys):
    head = create_table()
    for key in keys:
        index = key%size
        count = 0
        while get_node(head,index).key!=-1 and count<size:
            index = (index+1)%size
            count+=1
        if count<size:
            get_node(head,index).key = key
        else:
            print("Table Full:",key)
    print("Linear Probing:")
    for i in range(size):
        print(i,":",get_node(head,i).key)

def quadratic(keys):
    head = create_table()
    for key in keys:
        index = key%size
        inserted = False
        for i in range(size):
            pos = (index+i*i)%size
            if get_node(head,pos).key==-1:
                get_node(head,pos).key = key
                inserted = True
                break
        if not inserted:
            print("Could not insert:",key)
    print("Quadratic Probing:")
    for i in range(size):
        print(i,":",get_node(head,i).key)

n = int(input("Enter number of elements: "))
keys = []
for i in range(n):
    keys.append(int(input("Enter key: ")))
print("1. Separate Chaining")
print("2. Linear Probing")
print("3. Quadratic Probing")
```

```
ch = int(input("Enter choice: "))
if ch==1:
    chaining(keys)
elif ch==2:
    linear(keys)
elif ch==3:
    quadratic(keys)
else:
    print("Invalid Choice")
```

OUTPUT:

```
>>>
======== RESTART: /Users/karthikreddy/Desktop/SEM 3/DSA/lab/week 10/b.py =======
Enter number of elements: 4
Enter key: 12
Enter key: 18
Enter key: 20
Enter key: 14
1. Separate Chaining
2. Linear Probing
3. Quadratic Probing
Enter choice: 2
Linear Probing:
0 : 20
1 : -1
2 : 12
3 : -1
4 : 14
5 : -1
6 : -1
7 : -1
8 : 18
9 : -1
```

EXPLANATION:

- Defines a Node class to store a key and a reference to the next node, and creates a linked list of empty nodes for array-like hash table storage.
- The create_table function creates a linked list containing 10 empty nodes, while get_node retrieves a node at a specified index.
- The chaining function creates separate linked lists for each hash index and inserts colliding keys into the same bucket.
- The linear function resolves collisions by checking the next available node sequentially and inserts the key into an empty position.
- The quadratic function resolves collisions by checking positions using the square of the probing number and inserts the key into an empty node.
- The main program accepts keys, displays a menu and executes the selected hashing method.

ERROR TABLE:

S. No	Error Obtained	Rectification
1	Attribute error when get_node reaches an invalid node	Ensure the requested index is within the linked list size
2	Table full message appears when no empty node is available during linear probing	Limit probing attempts to the table size before insertion
3	Quadratic probing fails to insert a key into an available position	Use a suitable table size and verify the probing sequence